
GPU-Accelerated Machine Learning

on macOS, Linux and Windows

A complete, step-by-step guide to configuring TensorFlow, PyTorch and Keras to train on your GPU — Apple's Metal and MPS stack on a Mac, and the NVIDIA CUDA and cuDNN stack on Linux and Windows.

TensorFlow 2.19 – 2.21

PyTorch 2.13

Keras 3.15

Metal · MPS

CUDA 12 · 13

WSL2

✓ HOW THIS GUIDE WAS PREPARED

The macOS instructions were executed end to end and confirmed working on an Apple M4 (10-core GPU, 24 GB unified memory) running macOS 15.7.5. The Linux and Windows instructions were compiled from current NVIDIA, TensorFlow and PyTorch documentation and package indexes as of August 2026, and were not run on this machine — verify with the checks in each section.

00 How to Use This Guide

Training a neural network on a CPU can take hours where a GPU takes minutes. This guide gets your GPU working with TensorFlow, PyTorch and Keras — whichever operating system you are on. You do not need to read all of it: **read Part I, then jump to the one part that matches your machine.**

Who this guide is for

Anyone who wants to train machine-learning models on a GPU instead of a CPU, on a Mac, a Linux PC or a Windows PC. No prior systems knowledge is assumed. Every command can be copied and pasted directly into a terminal, and every step explains what it does and why it is needed.

Find your path

YOUR MACHINE	GPU STACK	READ	WHAT TO EXPECT
Mac with Apple Silicon M1 / M2 / M3 / M4	Metal + MPS built into macOS	Part I → Part II	The simplest setup. Nothing to install outside Python.
Linux with NVIDIA GPU	CUDA + cuDNN	Part I → Part III	The best-supported platform for deep learning. Driver, then packages.
Windows with NVIDIA GPU	CUDA + cuDNN natively or via WSL2	Part I → Part IV	PyTorch works natively; TensorFlow needs WSL2. Part IV explains the choice.
Intel Mac	None available	—	No supported GPU path. Use the CPU, or a cloud GPU service.
Linux with AMD GPU	ROCm	Part III , final section	Works with PyTorch on supported cards. Covered briefly.

What you will have at the end

- An isolated Python environment holding your machine-learning packages
- PyTorch training on the GPU — via **MPS** on a Mac, **CUDA** on Linux and Windows
- TensorFlow and Keras training on the GPU
- A Jupyter kernel wired to that environment
- A verification script that proves the GPU is genuinely being used, not silently skipped

ONE HABIT WORTH FORMING NOW

On every platform, both frameworks fall back to the CPU *silently* when GPU support is missing. Your code still runs — just far slower, with no error. Never assume the GPU is working; always run the verification step for your platform.

Contents

I Understanding the landscape

01 How GPU acceleration actually works

02 What you do and do not install

II macOS — Apple Silicon

03 Checking your Mac

04 Installation, step by step

05 Verifying the GPU works

III Linux — NVIDIA

06 Checking your Linux system

07 Installing the NVIDIA driver

08 Installing the frameworks

09 Verifying, and AMD ROCm

IV Windows — NVIDIA

10 Native Windows or WSL2?

11 The native Windows path

12 The WSL2 path

V Everyday use — all platforms

13 Writing portable GPU code

14 Jupyter and VS Code

15 Performance expectations

16 Limitations by platform

17 Troubleshooting

18 Quick reference card

Understanding the Landscape

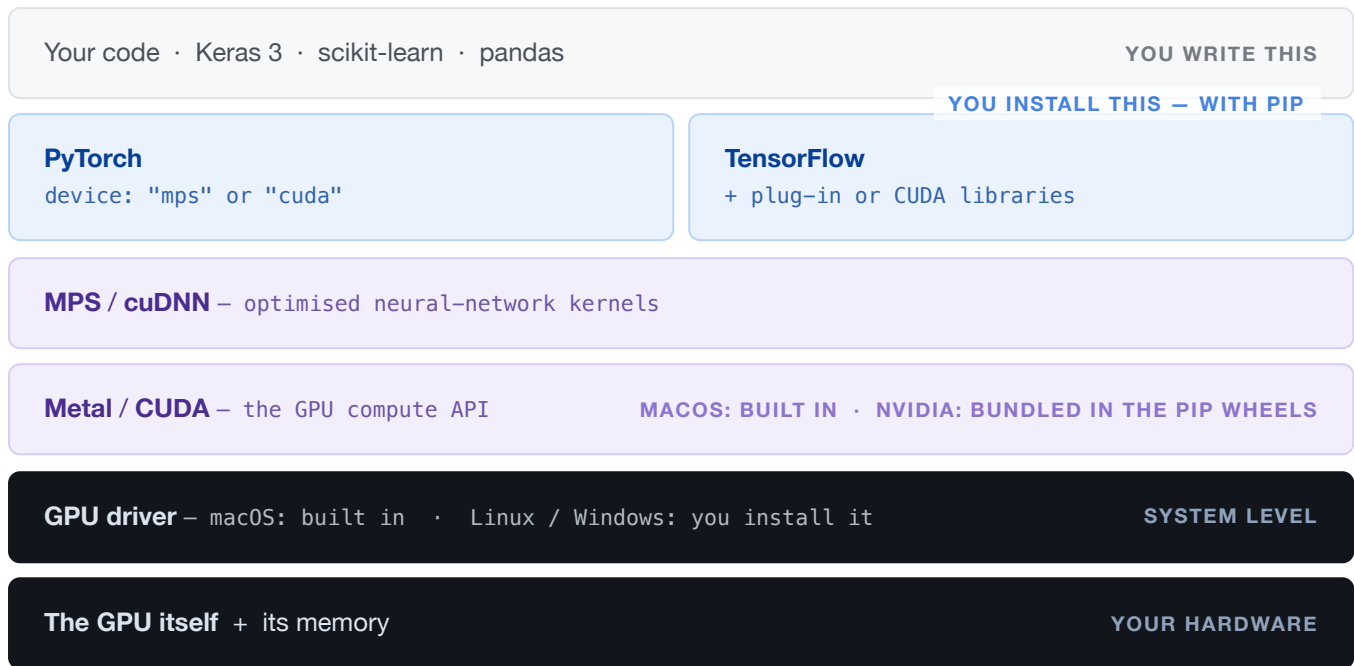
What a GPU stack is made of, why the pieces differ between Apple and NVIDIA, and — most importantly — how much of it you actually have to install yourself. Read this part whichever machine you are on; it is short.

01 How GPU Acceleration Actually Works

A GPU cannot run Python. For TensorFlow or PyTorch to use one, four layers must all be present and compatible. Understanding these four layers is what turns GPU setup from guesswork into a checklist.

LAYER	ON NVIDIA	ON APPLE SILICON	WHAT IT DOES
1. Driver	NVIDIA display driver	Part of macOS	Lets the operating system talk to the GPU at all. The only piece you install system-wide.
2. Compute platform	CUDA Toolkit	Metal	The general-purpose API for running arbitrary computation on the GPU.
3. Neural-net kernels	cuDNN	MPS	Hand-optimised implementations of convolution, matrix multiply and activations.
4. Framework	PyTorch · TensorFlow · Keras		What your Python code actually calls.

The stack, side by side



02 What You Do and Do Not Install

This is the single most useful thing to know before you begin, and the thing most older tutorials get wrong: **you almost certainly do not need to download the CUDA Toolkit or cuDNN from NVIDIA.**

For many years, setting up a GPU for deep learning meant downloading a multi-gigabyte CUDA Toolkit installer, then separately registering with NVIDIA to download cuDNN, unzipping it into the CUDA directory, and editing your `PATH`. Countless tutorials still describe this. It is no longer necessary for normal use.

Modern PyTorch and TensorFlow wheels **bundle the CUDA and cuDNN libraries they need inside the Python package itself**. When you `pip install` them, the right versions arrive automatically, matched to that build. This means:

✓ What you actually install

- **The GPU driver** — system-wide, once, on Linux and Windows. Nothing on macOS.
- **Python packages** — inside a virtual environment, with pip.

X What you can skip

- The **CUDA Toolkit installer** from NVIDIA's website.
- The **cuDNN download** and the manual file-copying that goes with it.
- Setting `PATH`, `LD_LIBRARY_PATH` or `CUDA_HOME` by hand.

WHEN YOU WOULD STILL NEED THE FULL CUDA TOOLKIT

Only if you are compiling GPU code yourself — building a custom CUDA extension, compiling a library from source, or using a tool that invokes the `nvcc` compiler. Ordinary training and inference never require it.

Two version numbers people confuse

`nvidia-smi` reports a CUDA version, and so does your PyTorch build. They mean different things, and they do not have to match.

NUMBER	WHAT IT MEANS
CUDA version in <code>nvidia-smi</code>	The <i>highest</i> CUDA version your installed driver can support. It is a ceiling, not what you have installed.
CUDA version in your wheel e.g. <code>cu130</code> in the <code>install</code> URL	The CUDA runtime actually bundled with PyTorch. This must be <i>less than or equal to</i> the driver's ceiling.

So a driver reporting `CUDA Version: 13.0` runs a `cu126` build of PyTorch perfectly well — newer drivers are backward compatible. The rule is simply: **keep your driver recent enough, and the pip wheels handle the rest.**

Minimum driver versions

CUDA FAMILY	MINIMUM NVIDIA DRIVER	NOTES
CUDA 12.x	525 or newer	Widely compatible. A safe default on older systems.
CUDA 13.x	580 or newer	Required for the newest PyTorch builds (<code>cu130</code> , <code>cu132</code>).

If in doubt, install the latest driver your distribution offers. Drivers are backward compatible, so a newer driver never breaks an older framework build.

Apple Silicon Macs

For any Mac with an M1, M2, M3, M4 or later chip. This is the shortest path in the guide — the GPU stack is already installed as part of macOS, so there is no driver and no toolkit to download.

IF YOU ARE COMING FROM AN NVIDIA MACHINE

There is no CUDA on a Mac and nothing to download from NVIDIA. Apple's Metal fills the role of the CUDA Toolkit and MPS fills the role of cuDNN, and both ship inside macOS. The only real difference from a Linux setup is that TensorFlow needs one extra package — `tensorflow-metal` — to reach the GPU.

03 Checking Your Mac

Open **Terminal** (press `⌘ + Space`, type "Terminal", press Return) and run the command below. It tells you your chip, your macOS version and how many GPU cores you have.

TERMINAL

```
sysctl -n machdep.cpu.brand_string  
sw_vers  
system_profiler SPDisplaysDataType | grep -E "Chipset|Cores|Metal"
```

Expected output (this machine)

Chip	Apple M4
Architecture	arm64 (Apple Silicon)
macOS	15.7.5 (Sequoia)
GPU cores	10
Metal support	Metal 3
Unified memory	24 GB

Requirements checklist

REQUIREMENT	NEEDED	WHY IT MATTERS
Chip	Apple Silicon (M-series)	Intel Macs cannot use this stack. Check that <code>uname -m</code> prints <code>arm64</code> .
macOS	12.0 or newer	Metal plug-in requirement. macOS 14+ recommended.
Python	3.10 – 3.12	Critical. See the warning below.
Free disk space	~6 GB	Frameworks and their dependencies are large.

⚠ PYTHON 3.12 IS THE MAXIMUM — DO NOT USE 3.13 OR 3.14

The `tensorflow-metal` plug-in ships pre-compiled wheels only for Python 3.9 through 3.12. Its most recent release (version 1.2.0, January 2025) has not been updated since. If you try to install it on Python 3.13 or newer, pip will simply report that no matching distribution was found.

This is precisely why step 1 of the installation creates a dedicated Python 3.12 environment rather than using whatever `python3` happens to point at.

Install the tooling

This guide uses `uv`, a fast Python package manager that can download and manage Python versions for you. If you do not already have it, install it with Homebrew:

TERMINAL

```
# Homebrew first, if you don't have it
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# then uv
brew install uv
```

PREFER CONDA?

Everything here works with conda too. Replace step 1 with `conda create -n ml-metal python=3.12 -y` and `conda activate ml-metal`, then use plain `pip install` for the remaining steps. Keep using the `pip` packages named below — do *not* install TensorFlow from conda-forge, as those builds do not include the Metal plug-in hooks.

04 Installation, Step by Step

Four commands. Roughly five to ten minutes, most of which is downloading. Run each one, wait for it to finish, then move to the next.

1 Create an isolated Python 3.12 environment

Keeping machine-learning packages in their own environment prevents version conflicts with anything else on your system, and lets you pin Python to 3.12 regardless of what is installed globally.

TERMINAL

```
uv venv --python 3.12 ~/.venvs/ml-metal
```

If Python 3.12 is not on your machine, uv downloads it automatically. The environment lives in a hidden folder in your home directory and can be deleted at any time with `rm -rf ~/.venvs/ml-metal`.

2 Activate the environment

TERMINAL

```
source ~/.venvs/ml-metal/bin/activate
```

Your prompt will change to show `(ml-metal)` at the front. That prefix is how you know the environment is active. **You must run this in every new Terminal window** before working on a project. To leave it, type `deactivate`.

3 Install the frameworks

TERMINAL

```
uv pip install "tensorflow==2.19.1" "tensorflow-metal==1.2.0" \
    torch torchvision torchaudio \
    jupyterlab matplotlib pandas scikit-learn
```

What each package is for:

PACKAGE	ROLE
<code>tensorflow</code>	The framework itself. The macOS ARM wheel is CPU-only on its own.
<code>tensorflow-metal</code>	This is the GPU part. Apple's PluggableDevice plug-in that routes TensorFlow operations onto the Metal GPU. Without it, TensorFlow silently runs on CPU.
<code>torch</code>	PyTorch. The standard wheel already contains the MPS GPU backend — there is no separate package to install.
<code>torchvision</code> , <code>torchaudio</code>	Datasets, model zoos and transforms for images and audio.
<code>keras</code>	Installed automatically as a TensorFlow dependency (Keras 3).
<code>jupyterlab</code> and others	Notebook interface plus the standard data-science toolkit.

⚠ PIN THESE TWO VERSIONS EXACTLY

`tensorflow-metal` is compiled against a specific TensorFlow plug-in interface. Apple officially pairs version 1.2.0 with TensorFlow 2.18, and it has been confirmed working with 2.19.1. Newer TensorFlow releases (2.20, 2.21) are *not* covered by any updated plug-in and may fail to register the GPU or crash inside kernels. If you hit trouble, drop to the officially supported pair:

```
uv pip install "tensorflow==2.18.1" "tensorflow-metal==1.2.0"
```

4 Register a Jupyter kernel

This makes the environment selectable from any Jupyter notebook or VS Code.

TERMINAL

```
~/venvs/ml-metal/bin/python -m ipykernel install --user --name ml-metal \
  --display-name "Python 3.12 (Metal GPU)"
```

✓ INSTALLATION COMPLETE

Now confirm that the GPU is actually being used — do not skip section 05.

05 Verifying the GPU Works

Both frameworks fall back to the CPU silently if GPU support is missing. That means your code will still run — just several times slower, with no error message. Always verify.

The quick check

TERMINAL

```
python -c "import torch; print('PyTorch MPS:', torch.backends.mps.is_available())"
python -c "import tensorflow as tf; print('TF GPUs:',
tf.config.list_physical_devices('GPU'))"
```

What you should see

```
PyTorch MPS: True
Metal device set to: Apple M4
TF GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

The line `Metal device set to: Apple M4` is printed by the Metal plug-in itself. If you see it, the plug-in has loaded successfully. If TF GPUs prints an empty list `[]`, the plug-in is not active — go to section 17.

The full verification script

Save the following as `verify_gpu.py` and run it with `python verify_gpu.py`. It confirms both frameworks, benchmarks the GPU against the CPU, and trains a small Keras model end to end.

VERIFY_GPU.PY

```

import time, numpy as np

# ----- PyTorch -----
import torch
print("torch", torch.__version__, "| MPS available:", torch.backends.mps.is_available())

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
a = torch.randn(4096, 4096, device=device)
b = torch.randn(4096, 4096, device=device)

torch.mps.synchronize()          # wait for GPU before timing
t = time.time()
for _ in range(20):
    c = a @ b
torch.mps.synchronize()          # wait again before stopping
print(f"20 matmuls on GPU: {time.time() - t:.3f}s")

# ----- TensorFlow / Keras -----
import tensorflow as tf, keras
print("tf", tf.__version__, "| keras", keras.__version__)
print("GPUs:", tf.config.list_physical_devices('GPU'))

model = keras.Sequential([
    keras.layers.Input(shape=(64,)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(1),
])
model.compile(optimizer='adam', loss='mse')
model.fit(np.random.rand(2000, 64), np.random.rand(2000, 1), epochs=2, verbose=0)
print("Keras training OK →", model.predict(np.random.rand(4, 64), verbose=0).shape)

```

Confirmed output on an Apple M4

```

Metal device set to: Apple M4
systemMemory: 24.00 GB
maxCacheSize: 8.00 GB

torch 2.13.0 | MPS available: True
20 matmuls on GPU: 1.968s
tf 2.19.1 | keras 3.15.1
GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
Keras training OK → (4, 1)

```

Watching the GPU while training

Two ways to see live GPU activity while a model is training:

Activity Monitor

Open Activity Monitor, then choose **Window** → **GPU History** (`⌘ + 4`). The graph should climb noticeably once training begins.

Terminal

Run in a second window:

```
sudo powermetrics --samplers gpu_power  
-i 1000
```

Watch the `GPU HW active residency` figure.

Linux with an NVIDIA GPU

The best-supported platform in deep learning: every framework targets it first, and almost all published research code assumes it. Three things to do — install the driver, create an environment, install the packages.

06 Checking Your Linux System

Before installing anything, confirm the machine actually has an NVIDIA GPU and find out whether a driver is already present. Open a terminal and run:

TERMINAL

```
# Is there an NVIDIA GPU in this machine at all?
lspci | grep -i nvidia

# Is a driver already installed and working?
nvidia-smi
```

If `lspci` prints a line mentioning your graphics card, the hardware is there. If `nvidia-smi` prints a table, you already have a working driver and can skip straight to section 08. If it reports `command not found`, continue to section 07.

Reading the `nvidia-smi` output

This table is the single most useful diagnostic on an NVIDIA system:

```
+-----+
| NVIDIA-SMI 580.65.06      Driver Version: 580.65.06      CUDA Version: 13.0      |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf   Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|   0   NVIDIA GeForce RTX 4070    | 00000000:01:00.0  On  |           N/A       |
| 30%   42C    P8      15W / 200W |    512MiB / 12282MiB |      0%      Default |
+-----+-----+-----+-----+-----+-----+

```

FIELD	WHAT IT TELLS YOU
Driver Version	What you have installed. Compare against the minimums in section 02.
CUDA Version	The <i>highest</i> CUDA your driver supports — a ceiling, not an installation. See section 02.
Memory-Usage	Your VRAM budget. This is the hard limit on model and batch size.
GPU-Util	Percentage busy. Watch this during training to confirm the GPU is really working.

Check your Python version

TERMINAL

```
python3 --version
```

TensorFlow 2.21 supports Python 3.10 – 3.13; PyTorch 2.13 supports 3.10 and newer. **Python 3.12 is a safe choice for both.** If your system Python is older or newer, section 08 shows how to get a specific version without touching the system one.

07 Installing the NVIDIA Driver

This is the one piece of software you install system-wide. Everything else lives inside a Python environment. Skip this section if `nvidia-smi` already works.

⚠️ INSTALL THE DRIVER ONLY — NOT THE CUDA TOOLKIT

You need the **driver**. You do **not** need NVIDIA's CUDA Toolkit installer: the CUDA and cuDNN libraries arrive bundled inside the PyTorch and TensorFlow pip packages, as explained in section 02. Installing the toolkit separately is a common source of version conflicts.

Ubuntu and Debian

Ubuntu can detect your card and recommend a driver automatically. This is the easiest and safest route:

TERMINAL

```
sudo apt update

# see which drivers are available and which is recommended
ubuntu-drivers devices

# install the recommended one automatically
sudo ubuntu-drivers install

# – or – pick a specific version, e.g. the 580 series for CUDA 13
sudo apt install nvidia-driver-580

sudo reboot
```

Fedora and RHEL

NVIDIA drivers come from the RPM Fusion repository:

TERMINAL

```
sudo dnf install \
  https://mirrors.rpmfusion.org/free/fedora/rpmfusion-free-release-$(rpm -E
%fedora).noarch.rpm \
  https://mirrors.rpmfusion.org/nonfree/fedora/rpmfusion-nonfree-release-$(rpm -E
%fedora).noarch.rpm

sudo dnf install akmod-nvidia
sudo reboot
```

Wait a few minutes after installing before rebooting — the kernel module is compiled in the background, and rebooting too early leaves you without a driver.

Arch Linux

TERMINAL

```
sudo pacman -S nvidia nvidia-utils
sudo reboot
```

Confirm it worked

After rebooting, `nvidia-smi` should print the table shown in section 06.

⚠ IF `NVIDIA-SMI` STILL FAILS AFTER A REBOOT

The usual culprit is **Secure Boot**, which blocks unsigned kernel modules. Either enrol the driver's signing key when prompted during installation (Ubuntu calls this MOK enrolment and asks you to set a password, then presents a blue screen on the next boot), or disable Secure Boot in your BIOS. The other common cause is that the open-source `nouveau` driver is still loaded — the official driver packages normally blacklist it for you, but a stale `initramfs` can keep it active.

08 Installing the Frameworks

1 Create an isolated environment

Never install machine-learning packages into your system Python — a bad upgrade can break system tools that depend on it. Use a virtual environment.

TERMINAL

```
# Option A – the standard library module (uses your system Python)
sudo apt install python3-venv          # Debian/Ubuntu only, if missing
python3 -m venv ~/.venvs/ml-gpu
source ~/.venvs/ml-gpu/bin/activate

# Option B – uv, which can also fetch a specific Python version for you
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv --python 3.12 ~/.venvs/ml-gpu
source ~/.venvs/ml-gpu/bin/activate
```

Your prompt will show `(ml-gpu)` once the environment is active. You must activate it in every new terminal window; `deactivate` leaves it.

2 Install PyTorch with CUDA

PyTorch publishes a separate build for each CUDA version, served from its own package index. The `--index-url` is what selects the GPU build — installing plain `pip install torch` on Linux gives you a CPU-only package.

TERMINAL

```
pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/cu130
```

Choose the build that matches your driver:

BUILD	NEEDS DRIVER	WHEN TO CHOOSE IT
<code>cu126</code>	525 or newer	Older drivers, or an older distribution you cannot upgrade.
<code>cu129</code>	575 or newer	Linux only — there is no Windows build of this one.
<code>cu130</code>	580 or newer	A good default on a current system.
<code>cu132</code>	580 or newer	The newest build, for the latest GPUs.

Substitute the name into the URL — for example `https://download.pytorch.org/whl/cu126`. If you are unsure, start with `cu126`: it has the widest driver compatibility and works on nearly everything.

3 Install TensorFlow with CUDA

TensorFlow uses a different mechanism: an optional dependency group, written in square brackets, that pulls in the NVIDIA libraries as pip packages.

TERMINAL

```
pip install "tensorflow[and-cuda]"
```

The quotes matter — without them, some shells (notably zsh) interpret the square brackets as a filename pattern and the command fails. The `[and-cuda]` suffix is what makes this a GPU install; plain `pip install tensorflow` gives you CPU-only.

4 Install the rest of your toolkit

TERMINAL

```
pip install jupyterlab matplotlib pandas scikit-learn

# register this environment as a Jupyter kernel
python -m ipykernel install --user --name ml-gpu \
    --display-name "Python (CUDA GPU)"
```

⚠ PYTORCH AND TENSORFLOW IN THE SAME ENVIRONMENT

Both packages bundle their own copies of the NVIDIA CUDA libraries, and they do not always agree on versions. Installing both together usually works, but when it does not, the symptom is confusing: one framework stops seeing the GPU after you install the other.

If that happens, the clean fix is **one environment per framework** — `~/.venvs/ml-torch` and `~/.venvs/ml-tf`. Disk space is cheap; debugging a library conflict is not.

09 Verifying, and a Note on AMD

Run these immediately after installing. Do not skip them.

TERMINAL

```
python -c "import torch; print('CUDA:', torch.cuda.is_available(), '|',  
torch.cuda.get_device_name(0))"  
python -c "import tensorflow as tf; print('GPUs:', tf.config.list_physical_devices('GPU'))"
```

What you should see

```
CUDA: True | NVIDIA GeForce RTX 4070  
GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

The full verification script

Save as `verify_gpu.py`. It reports your GPU, its memory, and times a large matrix multiplication so you can confirm real acceleration.

VERIFY_GPU.PY

```

import time
import torch

print("torch", torch.__version__, "| built for CUDA", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("device :", torch.cuda.get_device_name(0))
    total = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f"memory : {total:.1f} GB")

    a = torch.randn(8192, 8192, device="cuda")
    b = torch.randn(8192, 8192, device="cuda")

    for _ in range(3):
        c = a @ b
        torch.cuda.synchronize()
        # warm-up: first call includes setup cost
        # GPU calls are async – wait before timing

    t = time.time()
    for _ in range(20):
        c = a @ b
    torch.cuda.synchronize()
    dt = time.time() - t
    print(f"20 matmuls: {dt:.3f}s → {20 * 2 * 8192**3 / dt / 1e12:.1f} TFLOPS")

# TensorFlow, if installed
try:
    import tensorflow as tf
    print("tf", tf.__version__, "| GPUs:", tf.config.list_physical_devices('GPU'))
except ImportError:
    print("tensorflow not installed in this environment")

```

Watching the GPU during training

Open a second terminal and run one of these while your model trains. If `GPU-Util` stays near zero, your code is not actually using the GPU.

TERMINAL

```

# refresh the standard table every second
watch -n1 nvidia-smi

# or a compact live view of just the numbers that matter
nvidia-smi --query-gpu=utilization.gpu,memory.used,memory.total,temperature.gpu \
            --format=csv -l 1

```

For something friendlier, `pip install nvidia-top` then run `nvidia-top` — an interactive, colour-coded monitor.

AMD GPUs on Linux — ROCm

If your Linux machine has a modern AMD card rather than an NVIDIA one, PyTorch supports it through **ROCm**, AMD's equivalent of CUDA. Support is narrower than NVIDIA's but genuinely usable for PyTorch work.

TERMINAL

```
pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/rocm7.1
```

POINT	DETAIL
Linux only	The ROCm PyTorch builds are published for Linux x86-64 only — there is no Windows ROCm wheel.
Your code is unchanged	ROCm builds still use <code>torch.device("cuda")</code> and <code>torch.cuda.is_available()</code> . The CUDA names are kept deliberately so existing code runs untouched.
Check your card first	Only certain GPUs are officially supported. Consult AMD's ROCm compatibility list before committing time to it.
TensorFlow is harder	There is no equivalent one-line TensorFlow install. Use PyTorch, or AMD's container images.

Windows with an NVIDIA GPU

Windows has one important complication that no other platform has: PyTorch runs on the GPU natively, but TensorFlow no longer does. This part explains the choice and walks through both routes.

10 Native Windows or WSL2?

⚠ READ THIS BEFORE INSTALLING ANYTHING

TensorFlow dropped native Windows GPU support after version 2.10. TensorFlow 2.10 was the last release that could use an NVIDIA GPU directly on Windows. Every version since — including 2.19 and 2.21 — installs on Windows as **CPU-only**, no matter what GPU you own or which driver you have.

There is no flag or plug-in that restores this. A Windows TensorFlow wheel will install happily, run your code, and never touch your GPU. If you need TensorFlow on the GPU under Windows, you must use WSL2.

Your two options

Option A — Native Windows

PYTORCH ONLY

Install Python and PyTorch directly on Windows. Full GPU acceleration for PyTorch, and everything integrates naturally with Windows tools and file paths.

Choose this if you use PyTorch and do not need TensorFlow on the GPU. Most current deep-learning work — including nearly everything in the Hugging Face ecosystem — is PyTorch, so this suits many people.

Option B — WSL2

PYTORCH + TENSORFLOW

Windows Subsystem for Linux runs a real Ubuntu system inside Windows, with direct access to your NVIDIA GPU. You then follow the Linux instructions, and everything works — including TensorFlow.

Choose this if you need TensorFlow on the GPU, or you want your setup to match the Linux environments that most tutorials, research code and cloud servers assume.

CONSIDERATION	NATIVE WINDOWS	WSL2
PyTorch on GPU	Yes	Yes
TensorFlow on GPU	No — CPU only since 2.11	Yes
Setup effort	Lower	One extra step, then identical to Linux
Compatibility with tutorials	Occasional path and shell differences	Matches Linux exactly
Access to Windows files	Direct	Via <code>/mnt/c/</code> — but see the speed warning in section 12
Libraries needing compilation	Often awkward on Windows	Generally straightforward

RECOMMENDATION

If you only need PyTorch, take the native path in section 11 — it is simpler. If you need TensorFlow, or you are unsure, take WSL2 in section 12. WSL2 is also the better long-term choice: it removes an entire category of Windows-specific problems you would otherwise hit later.

11 The Native Windows Path

1 Install the NVIDIA driver

Download the Game Ready or Studio driver for your card from [nvidia.com/Download/index.aspx](https://www.nvidia.com/Download/index.aspx), or let the NVIDIA App update it for you. Either is fine — the Studio driver is slightly more conservative and is the usual choice for compute work. Restart when it finishes.

Then confirm it works. Open **PowerShell** and run:

POWERSHELL

```
nvidia-smi
```

You should see the table described in section 06. Remember that the CUDA version it reports is your driver's ceiling, not an installation.

DO NOT INSTALL THE CUDA TOOLKIT

The driver is all you need. PyTorch brings its own CUDA libraries.

2 Install Python

Get Python 3.12 from python.org/downloads. During installation, tick **"Add python.exe to PATH"** on the first screen — forgetting this is the most common Windows Python problem.

AVOID THE MICROSOFT STORE VERSION

The Store build of Python runs in a sandbox with redirected file paths, which causes confusing permission and path errors with scientific packages. Use the installer from python.org.

POWERSHELL

```
python --version
py --list      # shows every Python version installed
```

3 Create and activate an environment

POWERSHELL

```
cd $HOME
py -3.12 -m venv ml-gpu
.\ml-gpu\Scripts\Activate.ps1
```

Note the Windows activation path — `Scripts\Activate.ps1`, not `bin/activate`. In the older Command Prompt it is `ml-gpu\Scripts\activate.bat` instead.

⚠ "RUNNING SCRIPTS IS DISABLED ON THIS SYSTEM"

PowerShell blocks scripts by default. Allow them for your own account with:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

4 Install PyTorch with CUDA

POWERSHELL

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu130
```

As on Linux, the `--index-url` selects the GPU build. Windows builds exist for `cu126`, `cu130` and `cu132` — note that `cu129` is **Linux-only**, so do not copy that URL from a Linux tutorial. Use `cu126` if your driver is older than 580.

5 Install everything else, and verify

POWERSHELL

```
pip install jupyterlab matplotlib pandas scikit-learn

python -c "import torch; print(torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

A result of `True` followed by your card's name means you are done. The full verification script in section 09 works identically on Windows.

What about TensorFlow on native Windows?

You have three choices, none of them ideal:

APPROACH	REALITY
Use WSL2 section 12	The recommended answer. Full GPU support and the current TensorFlow version.
CPU-only TensorFlow	<code>pip install tensorflow-cpu</code> . Fine for learning the API, small models and inference. Honest about what it is.
Pin TensorFlow 2.10	The last native-GPU release, from 2022. It needs Python 3.7 – 3.10 and a manual CUDA 11.2 / cuDNN 8.1 installation. Not recommended: you lose Keras 3 and years of fixes.

12 The WSL2 Path

WSL2 runs a genuine Ubuntu system inside Windows with direct access to your NVIDIA GPU. Once it is set up, you follow Part III exactly — and TensorFlow works on the GPU as it does on any Linux box.

1 Install the Windows NVIDIA driver — and only that

Install the normal Windows driver as described in section 11, step 1. Modern NVIDIA drivers include WSL2 GPU support automatically.

⚠ THE SINGLE MOST COMMON WSL2 MISTAKE

Never install an NVIDIA driver inside WSL. Do not run `apt install nvidia-driver-580` in your Ubuntu shell. The Linux side reaches your GPU through the *Windows* driver, and installing a Linux driver on top of that breaks the connection — usually producing an `nvidia-smi` that no longer works inside WSL.

Inside WSL you install the CUDA-enabled *Python packages* only. The driver stays on the Windows side, and only there.

2 Install WSL2 and Ubuntu

Open **PowerShell as Administrator** and run:

POWERSHELL (ADMINISTRATOR)

```
wsl --install
# then restart Windows when prompted
```

This installs WSL2 with Ubuntu by default. After restarting, Ubuntu opens and asks you to create a username and password — these are separate from your Windows account, and the password is what `sudo` will ask for.

Useful commands from PowerShell afterwards:

POWERSHELL

```
wsl --list --verbose    # check it is running version 2, not 1
wsl --update           # update the WSL kernel
wsl                   # open your Linux shell
```

3 Confirm the GPU is visible inside Linux

In the Ubuntu terminal, run:

UBUNTU (INSIDE WSL)

```
nvidia-smi
```

This should print the same table you saw in Windows. It works because WSL2 passes the GPU through from the Windows driver — nothing was installed on the Linux side to make it happen. **If this fails, stop and fix it before going further:** update your Windows driver and run `wsl --update`, and make sure you did not install a Linux driver in step 1.

4 Install the frameworks

From here you are on Linux, so Part III applies exactly. In brief:

```

sudo apt update && sudo apt install -y python3-venv python3-pip

python3 -m venv ~/.venvs/ml-gpu
source ~/.venvs/ml-gpu/bin/activate

# PyTorch
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu130

# TensorFlow – the GPU build that native Windows cannot give you
pip install "tensorflow[and-cuda]"

pip install jupyterlab matplotlib pandas scikit-learn

```

Then verify with the commands in section 09.

Living with WSL2

TOPIC	WHAT TO KNOW
Keep projects in the Linux home directory	Store code and datasets under <code>~/</code> , not <code>/mnt/c/</code> . Crossing the Windows file-system boundary is dramatically slower — enough to make data loading, not your GPU, the bottleneck. This is the most impactful WSL2 tip there is.
Reaching Windows files	Your C: drive is at <code>/mnt/c/</code> . Fine for occasional copying; poor for a training data directory.
Reaching Linux files from Windows	Type <code>\\wsl\$</code> into File Explorer, or run <code>explorer.exe .</code> from your Ubuntu shell to open the current folder.
Jupyter	Run <code>jupyter lab</code> inside WSL and open the printed <code>localhost</code> link in your normal Windows browser. Networking is forwarded automatically.
VS Code	Install the WSL extension, then run <code>code .</code> from the Ubuntu shell. VS Code runs its interface on Windows and its Python machinery inside Linux — the recommended way to work.
Memory limits	WSL2 claims up to half your system RAM by default. To change it, create <code>C:\Users\<you>\.wslconfig</code> with a <code>[wsl2]</code> section and a <code>memory=16GB</code> line, then run <code>wsl --shutdown</code> .

Everyday Use

Writing code that runs on any machine, working in notebooks, knowing what performance to expect, and fixing things when they break. This part applies whichever operating system you installed on.

13 Writing Portable GPU Code

Write your code once and it will use whatever GPU is present — CUDA on a Linux or Windows machine, MPS on a Mac, and the CPU anywhere else. This costs three extra lines and saves endless trouble when you move between machines.

PyTorch — you must move things explicitly

PyTorch never moves data to the GPU on your behalf. Both the model and every input tensor must be sent to the device. Start every project with this helper:

```
import torch

def get_device():
    """Pick the best available device on any machine."""
    if torch.cuda.is_available():          # NVIDIA on Linux/Windows, or AMD ROCm
        return torch.device("cuda")
    if torch.backends.mps.is_available():  # Apple Silicon
        return torch.device("mps")
    return torch.device("cpu")            # everything else

device = get_device()
print("training on:", device)
```

Then use it consistently — the model once, and every batch as it arrives:

```

model = MyNetwork().to(device)                # move the model once

for inputs, labels in dataloader:
    inputs = inputs.to(device)                 # move each batch
    labels = labels.to(device)

    optimizer.zero_grad()
    loss = criterion(model(inputs), labels)
    loss.backward()
    optimizer.step()

results = predictions.cpu().numpy()            # bring results back for NumPy

```

⚠ THE MOST COMMON PYTORCH ERROR, ON EVERY PLATFORM

Expected all tensors to be on the same device (CUDA), or
Placeholder storage has not been allocated on MPS device (Mac), both mean the same thing: you moved the model but forgot an input tensor, or the other way round. Both sides of every operation must live on the same device.

Synchronising before you time anything

GPU calls are asynchronous — Python returns immediately while the GPU is still working. Timing code without synchronising first produces meaningless results, usually implausibly fast ones.

```

torch.cuda.synchronize()    # NVIDIA / ROCm
torch.mps.synchronize()     # Apple Silicon

```

TensorFlow and Keras — automatic

TensorFlow places operations on the GPU by default once GPU support is present — the `tensorflow-metal` plug-in on a Mac, or the bundled CUDA libraries on Linux. Your existing code needs no changes at all, on any platform.

```

import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([...])
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=10)  # already on the GPU

```

Forcing CPU when you need to

Useful for debugging, or for the small models that are genuinely faster on CPU (see section 15):

```
# Option A – for one block of code
with tf.device('/CPU:0'):
    model.fit(X_train, y_train, epochs=10)

# Option B – for a whole script, set before importing tensorflow
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
```

Limiting GPU memory growth

NVIDIA

By default TensorFlow claims almost all of your VRAM the moment it starts, which stops you running a second notebook or sharing the card. This is a common annoyance on NVIDIA systems and is worth setting in every script:

```
import tensorflow as tf

for gpu in tf.config.list_physical_devices('GPU'):
    tf.config.experimental.set_memory_growth(gpu, True) # allocate as needed
```

Keras 3 with a PyTorch backend

Keras 3 can run on top of PyTorch instead of TensorFlow, which gives you Keras's API over whatever PyTorch supports — CUDA or MPS. This is especially useful on Windows, where it is the only way to get Keras onto the GPU without WSL2. Set the environment variable before importing Keras:

```
import os
os.environ["KERAS_BACKEND"] = "torch" # must come before the import
import keras
```

A GENUINELY USEFUL TRICK FOR WINDOWS USERS

Native Windows cannot give TensorFlow a GPU — but it can give PyTorch one. Running Keras 3 on the PyTorch backend means you can write ordinary Keras code and still train on your GPU, without installing WSL2. Not every TensorFlow-specific feature carries over, but for standard Keras models it works well.

14 Using It in Jupyter and VS Code

Activating your environment

The command differs by platform. This is the one thing you will type every day:

PLATFORM	COMMAND
macOS / Linux / WSL2	<code>source ~/.venvs/ml-gpu/bin/activate</code>
Windows PowerShell	<code>.\ml-gpu\Scripts\Activate.ps1</code>
Windows Command Prompt	<code>ml-gpu\Scripts\activate.bat</code>
Any — to leave	<code>deactivate</code>

On a Mac the environment built in Part II is named `ml-metal` rather than `ml-gpu`.

JupyterLab and VS Code

JupyterLab

Activate the environment, then:

```
jupyter lab
```

In a notebook, choose **Kernel** → **Change Kernel** and pick the kernel you registered with `ipykernel install`.

In WSL2: run this inside Ubuntu and open the printed `localhost` link in your Windows browser.

VS Code

Open a notebook or Python file, press `⌘ ⬆ P` (Mac) or `Ctrl ⬆ P` (Windows/Linux), run **Python: Select Interpreter**, and choose the interpreter inside your environment folder.

In WSL2: install the **WSL** extension and run `code .` from the Ubuntu shell — do not open the project through `\\wsl$` from Windows.

Confirming the GPU from inside a notebook

Worth running as the first cell of any notebook, so you never discover halfway through a long training run that you were on the CPU:

```
import torch
print("CUDA:", torch.cuda.is_available(), "| MPS:", torch.backends.mps.is_available())
```

Making activation less tedious

MACOS

LINUX

Add a shortcut to your shell profile so one short word activates the environment:

TERMINAL

```
# use ~/.zshrc on macOS, ~/.bashrc on most Linux systems and WSL2
echo 'alias mlgpu="source ~/.venvs/ml-gpu/bin/activate"' >> ~/.zshrc
source ~/.zshrc
```

From then on, simply type `mlgpu` in any new terminal window.

15 Realistic Performance Expectations

Set your expectations correctly and you will avoid a lot of frustration. A GPU is not a uniform multiplier — the speed-up depends enormously on what you are running, and on a Mac in particular the GPU does not win every workload.

What each platform is good for

PLATFORM	REALISTIC SPEED-UP	CHARACTER
NVIDIA discrete GPU Linux or Windows	10 – 50 × over CPU	The real thing. A mid-range card turns an overnight CPU job into a coffee break. Dedicated VRAM is the limit on model size.
Apple Silicon M-series	1 – 5 × over CPU	A genuine accelerator, but modest. Its real advantage is large unified memory and running silently on battery.

WHY THE MAC NUMBERS LOOK LOW

Apple's CPU cores contain dedicated matrix hardware (the AMX units) that NumPy and PyTorch reach through the Accelerate framework. The CPU baseline on a Mac is therefore unusually strong, which makes the GPU's relative gain look small. It is the CPU being fast, not the GPU being broken.

Measured on the machine this guide was prepared on

OPERATION	GPU (MPS)	CPU
20 × matrix multiply, 4096 × 4096	1.97 s	2.25 s (equivalent)

Apple M4, 10-core GPU, 24 GB unified memory, PyTorch 2.13. The same benchmark on a mid-range NVIDIA card would complete in a small fraction of this.

Where the GPU actually wins

WORKLOAD	NVIDIA	APPLE	NOTES
Convolutional networks (CNNs)	10 – 30 ×	2 – 5 ×	The strongest case on both. Image classification, segmentation.
Transformers, medium batches	10 – 40 ×	2 – 4 ×	Attention maps well onto both stacks.
LLM inference	10 – 50 ×	3 – 10 ×	On Apple, unified memory lets unusually large models fit.
Large dense matrix maths	10 – 30 ×	1 – 1.5 ×	Modest on Apple, as measured above.
Small models, tiny batches	Little gain	Often slower	Dispatch overhead exceeds the compute saved.
Classical ML (scikit-learn)	No effect		scikit-learn is CPU-only on every platform. Use cuML if you need GPU classical ML on NVIDIA.

ALWAYS BENCHMARK BEFORE ASSUMING

Time one epoch on GPU and one on CPU for your actual model before committing to either. Increasing the batch size is usually the single most effective way to make a GPU worth using — it gives the hardware enough work to offset the cost of dispatching it.

Memory: the limit that actually bites

In practice you run out of GPU memory long before you run out of speed. This is what caps your batch size and model size, and it behaves differently on the two platforms.

NVIDIA — dedicated VRAM

A fixed, separate pool: 8, 12, 16, 24 GB.

Exceeding it raises `CUDA out of memory`.

Fast, but a hard ceiling — and data must cross the PCIe bus to reach it.

Apple — unified memory

Shared with the whole system, so a 24 GB Mac can hold models that would not fit on a 12 GB card. No copying across a bus, but you are competing with macOS and every other open application.

If you hit "out of memory"

- **Reduce the batch size first.** It is the largest single lever, and halving it usually roughly halves memory use.
- **Use mixed precision** — 16-bit tensors nearly halve memory and run faster on modern NVIDIA cards.

- **Use gradient accumulation** to keep an effective large batch size while processing small ones.
- **Clear the cache between experiments** in notebooks: `torch.cuda.empty_cache()` or `torch.mps.empty_cache()`.
- **Restart the kernel.** In Jupyter, memory from a crashed run is often still held until you do.

Unified memory in more detail MACOS

Unlike a discrete GPU with its own fixed VRAM, Apple Silicon shares one pool of memory between CPU and GPU. This cuts both ways:

The advantage

Large models fit that would not fit on a comparable discrete card, and moving data between CPU and GPU is cheap because there is no copy across a PCIe bus.

The catch

You are sharing with macOS and every other running application. On a 24 GB machine, budget roughly 6 – 8 GB for the system and keep an eye on memory pressure in Activity Monitor.

16 Limitations by Platform

Every platform has constraints worth knowing before you hit them at an inconvenient moment. NVIDIA has the fewest, which is why it remains the default assumption in most published code.

NVIDIA on Linux LINUX

LIMITATION	WHAT IT MEANS AND WHAT TO DO
Driver and kernel coupling	The driver builds a kernel module, so a kernel upgrade can leave you without a working GPU until it rebuilds. Keep the <code>dkms</code> package installed so this happens automatically, and re-run <code>nvidia-smi</code> after any kernel update.
Secure Boot	Blocks unsigned kernel modules. Enrol the signing key (MOK) during driver installation, or disable Secure Boot in the BIOS.
Frameworks disagree on CUDA versions	PyTorch and TensorFlow each bundle their own CUDA libraries. In one shared environment they can conflict — use one environment per framework if a working install stops seeing the GPU after you add the other.
VRAM is a hard ceiling	Unlike Apple's unified memory, you cannot borrow system RAM. An 8 GB card simply cannot hold some models at any batch size.

LIMITATION	WHAT IT MEANS AND WHAT TO DO
No native TensorFlow GPU	The defining Windows limitation, covered in section 10. TensorFlow 2.10 was the last release with native Windows GPU support. Use WSL2, or run Keras 3 on the PyTorch backend.
No <code>cu129</code> build	That PyTorch build is Linux-only. Copying an index URL from a Linux tutorial will fail on Windows. Use <code>cu126</code> , <code>cu130</code> or <code>cu132</code> .
Libraries that need compiling	Packages without pre-built Windows wheels may need Visual Studio Build Tools. Under WSL2 this problem largely disappears.
<code>num_workers</code> in <code>DataLoader</code>	Windows spawns processes differently from Linux. Multi-worker data loading must be guarded by <code>if __name__ == "__main__":</code> or it will fail or hang.
WSL2 file-system speed	Reading training data from <code>/mnt/c/</code> is slow enough to bottleneck training. Keep datasets inside the Linux home directory.

LIMITATION	WHAT IT MEANS AND WHAT TO DO
No float64	MPS supports 32-bit and 16-bit floats only. Code that requests <code>torch.float64</code> or <code>dtype=np.float64</code> will raise a <code>TypeError</code> . Cast your data to <code>float32</code> — this is standard practice in deep learning anyway and costs no accuracy.
Some operations are unimplemented	A minority of PyTorch operators have no MPS kernel yet and raise <code>NotImplementedError</code>. Enable automatic CPU fallback so they run transparently instead of crashing: <pre>export PYTORCH_ENABLE_MPS_FALLBACK=1</pre>
No multi-GPU training	There is one GPU and it cannot be pooled with another machine's. <code>DistributedDataParallel</code> and <code>tf.distribute.MirroredStrategy</code> have nothing to distribute across.
No CUDA-only libraries	Anything that compiles CUDA kernels — <code>bitsandbytes</code> , <code>flash-attention</code> , <code>xformers</code> , <code>apex</code> , <code>deepspeed</code> , most custom CUDA extensions — will not work. Look for MPS-aware alternatives, or use MLX , Apple's own array framework, for hand-optimised work.
Mixed precision is limited	<code>torch.cuda.amp</code> does not apply. Use <code>torch.float16</code> or <code>torch.bfloat16</code> tensors directly. In Keras, <code>mixed_float16</code> works but the gain is smaller than on NVIDIA hardware.
The plug-in lags behind	<code>tensorflow-metal</code> is maintained by Apple and updated infrequently — version 1.2.0 dates from January 2025. Expect to stay a few TensorFlow releases behind the latest. PyTorch's MPS backend, by contrast, is part of PyTorch itself and tracks new releases closely.
Non-determinism	Some MPS kernels are not bit-reproducible between runs. If you need exact reproducibility for a paper or a regression test, run that job on the CPU.

IF PYTORCH IS ALL YOU NEED

PyTorch's MPS backend is more actively maintained and supports newer Python versions than the TensorFlow plug-in. If a project does not specifically require TensorFlow, a PyTorch-only environment can use Python 3.13 or newer and avoids the version pinning in step 3 entirely.

17 Troubleshooting

Start here, whatever the symptom

Nearly every "the GPU isn't working" report comes down to one of four things. Check them in this order before anything else:

#	CHECK	HOW
1	Is the environment actually active?	Your prompt should show the environment name in brackets. Installing into one environment and running in another is the single most common mistake.
2	Does the system see the GPU?	<code>nvidia-smi</code> on NVIDIA; on a Mac the GPU is always present.
3	Did you install the GPU build?	Plain <code>pip install torch</code> on Linux/Windows is CPU-only — you need <code>--index-url</code> . Plain <code>pip install tensorflow</code> on Linux needs <code>[and-cuda]</code> .
4	Is it a version mismatch?	Driver too old for the CUDA build, or Python too new for the package.

SYMPTOM	CAUSE AND FIX
<code>torch.cuda.is_available()</code> is <code>False</code>	Usually the CPU-only wheel. Check with <code>python -c "import torch; print(torch.version.cuda)"</code> — if it prints <code>None</code> , you have the CPU build. Reinstall using the <code>--index-url</code> from section 08. If it prints a version but CUDA is still unavailable, your driver is too old for that build.
<code>nvidia-smi: command not found</code>	No driver installed. See section 07 (Linux) or section 11 (Windows). Inside WSL2, this instead means the <i>Windows</i> driver is missing or too old — never install a driver inside WSL.
Failed to initialise NVML: Driver/library version mismatch	The driver was updated but the old kernel module is still loaded. Reboot. If it persists after a kernel upgrade, reinstall the driver so the module rebuilds.
CUDA out of memory	Reduce the batch size, use mixed precision, or call <code>torch.cuda.empty_cache()</code> . In notebooks, restart the kernel — a crashed run often still holds memory. Check for stray processes with <code>nvidia-smi</code> .
TensorFlow finds no GPU on Windows	Expected — this is not a fault. Native Windows TensorFlow has been CPU-only since 2.11. See section 10.
TensorFlow finds no GPU on Linux	You most likely installed plain <code>tensorflow</code> instead of <code>"tensorflow[and-cuda]"</code> . Reinstall with the bracket suffix, quoted.
PyTorch sees the GPU but TensorFlow does not, in the same environment	A bundled-CUDA-library conflict. Use separate environments for the two frameworks — see the warning in section 08.
<code>nvidia-smi</code> works but shows 0 % use while training	Your code is on the CPU. In PyTorch, confirm both the model and the batches were moved with <code>.to(device)</code> . Also possible: data loading is the bottleneck — raise <code>num_workers</code> in your DataLoader.
WSL2: <code>nvidia-smi</code> works in Windows but not in Ubuntu	Run <code>wsl --update</code> in PowerShell, update the Windows driver, and confirm you did not install a Linux NVIDIA driver inside WSL. If you did, remove it.

SYMPTOM	CAUSE AND FIX
No matching distribution found for tensorflow-metal	Your Python is 3.13 or newer, or you are on an Intel Mac. Confirm with <code>python --version</code> and <code>uname -m</code> (must print <code>arm64</code>), then rebuild the environment with <code>--python 3.12</code> .
<code>tf.config.list_physical_devices('GPU')</code> returns <code>[]</code>	The plug-in is not installed in the <i>active</i> environment. Confirm your prompt shows <code>(ml-metal)</code> , then run <code>pip list grep metal</code> . Reinstall if it is missing.
<code>torch.backends.mps.is_available()</code> is <code>False</code>	Almost always an Intel Mac or macOS older than 12.3. Check <code>torch.backends.mps.is_built()</code> : if that is <code>False</code> too, the wheel is a non-Apple build — reinstall with <code>pip install --force-reinstall torch</code> .
Crashes or garbage results after upgrading TensorFlow	A TensorFlow / plug-in mismatch. Return to the confirmed pair: <code>uv pip install "tensorflow==2.18.1" "tensorflow-metal==1.2.0"</code> .
<code>NotImplementedError: ... not currently implemented for the MPS device</code>	Expected for a few rarer operators. Set <code>PYTORCH_ENABLE_MPS_FALLBACK=1</code> before running so those operators fall back to the CPU automatically.
Placeholder storage has not been allocated on MPS device	Model and data are on different devices. Ensure every input tensor gets <code>.to(device)</code> , not just the model.
Training is slower on GPU than on CPU	Usually correct behaviour, not a fault — see section 15. Increase the batch size; if it is still slower, use the CPU for that model.
Machine becomes unresponsive during training	Unified memory exhausted. Reduce the batch size, close other applications, and watch memory pressure in Activity Monitor.
Nothing else works	Delete and rebuild — it takes minutes and resolves most dependency tangles: <pre>rm -rf ~/.venvs/ml-metal</pre> then repeat section 04 from step 1.

18 Quick Reference Card

The install commands, side by side

PLATFORM	PYTORCH	TENSORFLOW
macOS Apple Silicon	<code>pip install torch torchvision torchaudio</code> MPS is built in	<code>pip install "tensorflow==2.19.1"</code> <code>"tensorflow-metal==1.2.0"</code>
Linux NVIDIA	<code>pip install torch torchvision torchaudio</code> <code>--index-url</code> <code>https://download.pytorch.org/whl/cu130</code>	<code>pip install "tensorflow[and-cuda]"</code>
Windows native	<code>pip install torch torchvision torchaudio</code> <code>--index-url</code> <code>https://download.pytorch.org/whl/cu130</code>	No GPU support. Use WSL2, or <code>pip install tensorflow-cpu</code> .
Windows WSL2	Identical to the Linux row above	
Linux AMD	<code>pip install torch torchvision torchaudio</code> <code>--index-url</code> <code>https://download.pytorch.org/whl/rocm7.1</code>	No simple path

Everyday commands

```
# activate – macOS / Linux / WSL2
source ~/.venvs/ml-gpu/bin/activate

# activate – Windows PowerShell
.\ml-gpu\Scripts\Activate.ps1

# leave
deactivate

# start notebooks
jupyter lab

# what's installed
pip list | grep -E "tensor|torch|keras"
```

GPU checks

```
# PyTorch – NVIDIA / AMD
torch.cuda.is_available()
torch.cuda.get_device_name(0)
torch.cuda.synchronize()
torch.cuda.empty_cache()

# PyTorch – Apple Silicon
torch.backends.mps.is_available()
torch.mps.synchronize()

# TensorFlow – everywhere
tf.config.list_physical_devices('GPU')

# the system view (NVIDIA)
nvidia-smi
```

Versions referenced in this guide

COMPONENT	VERSION	NOTE
torch	2.13.0	All platforms. CUDA and MPS support built into the wheels.
tensorflow	2.19.1 / 2.21.0	2.19.1 verified on macOS; 2.21 current on Linux.
tensorflow-metal	1.2.0	macOS only. Pin it — latest available.
keras	3.15.1	Installed with TensorFlow.
Python	3.12	Safe on every platform. Max 3.12 with <code>tensorflow-metal</code> .
CUDA builds	cu126 · cu129 · cu130 · cu132	<code>cu129</code> is Linux-only.
NVIDIA driver	525+ / 580+	525 for CUDA 12, 580 for CUDA 13.
macOS	15.7.5 (min 12.0)	Verified on Apple M4.

Environment variables worth knowing

VARIABLE	PLATFORM	EFFECT
<code>CUDA_VISIBLE_DEVICES=-1</code>	NVIDIA	Hides all GPUs, forcing CPU execution. Useful for debugging.
<code>CUDA_VISIBLE_DEVICES=0</code>	NVIDIA	Restricts your process to GPU 0 on a multi-GPU machine.
<code>PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True</code>	NVIDIA	Reduces memory fragmentation on long training runs.
<code>PYTORCH_ENABLE_MPS_FALLBACK=1</code>	macOS	Unsupported operators run on CPU instead of raising an error.
<code>KERAS_BACKEND=torch</code>	All	Runs Keras 3 on PyTorch — the Windows GPU workaround.
<code>TF_CPP_MIN_LOG_LEVEL=2</code>	All	Silences TensorFlow's informational startup messages.

Where to go next

RESOURCE	WHAT IT IS GOOD FOR
Apple — Metal for TensorFlow developer.apple.com/metal/tensorflow-plugin/	The official plug-in page. Check here first after any macOS or TensorFlow upgrade to see which pairing Apple currently supports.
PyTorch MPS documentation docs.pytorch.org/docs/stable/notes/mps.html	The authoritative list of what the MPS backend supports, plus the current set of unimplemented operators.
MLX github.com/ml-explore/mlx	Apple's own array framework, designed for Apple Silicon from the ground up. Worth considering for LLM work and for anything where the Metal plug-in feels limiting.
Keras 3 backend guide keras.io/getting_started/	How to switch Keras between the TensorFlow, PyTorch and JAX backends — the workaround for Keras on the GPU under native Windows.
PyTorch — Get Started Locally pytorch.org/get-started/locally/	An interactive selector that generates the exact install command for your OS, package manager and CUDA version. The authoritative source when the URLs in this guide age.
TensorFlow — pip install guide tensorflow.org/install/pip	Official per-platform instructions, including the Windows and WSL2 situation described in Part IV.
NVIDIA CUDA on WSL guide docs.nvidia.com/cuda/wsl-user-guide/	The definitive reference for GPU passthrough into WSL2, including why you must not install a driver inside WSL.

Maintaining this setup

All platforms

- **Re-run your platform's verification checks after any upgrade** — framework, driver or operating system. Silent CPU fallback is the failure mode to watch for, and it never announces itself.
- **Freeze your working set** with `pip freeze > requirements.txt` so you can rebuild the exact environment if an upgrade goes wrong.
- **Change one thing at a time.** Upgrading the driver and the framework together makes it impossible to tell which one broke your setup.

NVIDIA — Linux and Windows

- **Keep the driver reasonably current**, but there is no need to chase every release. Drivers are backward compatible, so a newer driver will not break an older framework build.
- **On Linux, check `nvidia-smi` after every kernel upgrade.** The driver module has to rebuild against the new kernel, and `dkms` is what makes that automatic.

- **Upgrading PyTorch's CUDA build** means reinstalling with a different `--index-url` — upgrading the package alone keeps the old CUDA version.

Apple Silicon

- **Do not upgrade TensorFlow casually.** Check Apple's plug-in page first — a new TensorFlow release without a matching `tensorflow-metal` update will usually break GPU support.
- **PyTorch upgrades are safe.** MPS is maintained inside PyTorch itself and improves with each release.

The macOS instructions were executed and verified on an Apple M4 running macOS 15.7.5 on 13 August 2026.
The Linux and Windows instructions follow current NVIDIA, PyTorch and TensorFlow documentation as of that date.
Package versions change quickly — always re-run the verification checks for your platform after an upgrade.